



SOVEREIGN AI WORKFORCE

Production Architecture Document

System Design · Data Architecture · Security · Scalability · Cost Model

Boukhalfa Fateh Abderrahim

Sovereign AI Engineer · USTHB · Algeria dz

USAII Global AI Hackathon 2026

ENGINEERING DOCUMENT — v1.0

Contents

1. System Overview
2. Frontend Architecture
3. Backend Architecture
4. Database Architecture
5. Vector Database Architecture
6. AI Orchestration Layer
7. Memory Layer
8. Authentication
9. Security Model
10. Monitoring
11. Observability
12. Scalability Strategy
13. Deployment Architecture
14. Cost Estimation

1. System Overview

Sovereign AI Workforce is a multi-tenant orchestration platform that coordinates specialized AI agents to execute organizational workflows, gated behind human approval, with long-term semantic memory and real Google Workspace integration. This document specifies the production architecture: the version of the system designed to serve hundreds of organizations reliably, securely, and cost-effectively — extending the working hackathon backend already implemented (FastAPI + PostgreSQL/pgvector + Docker) into an enterprise-scale deployment.

High-Level System Diagram

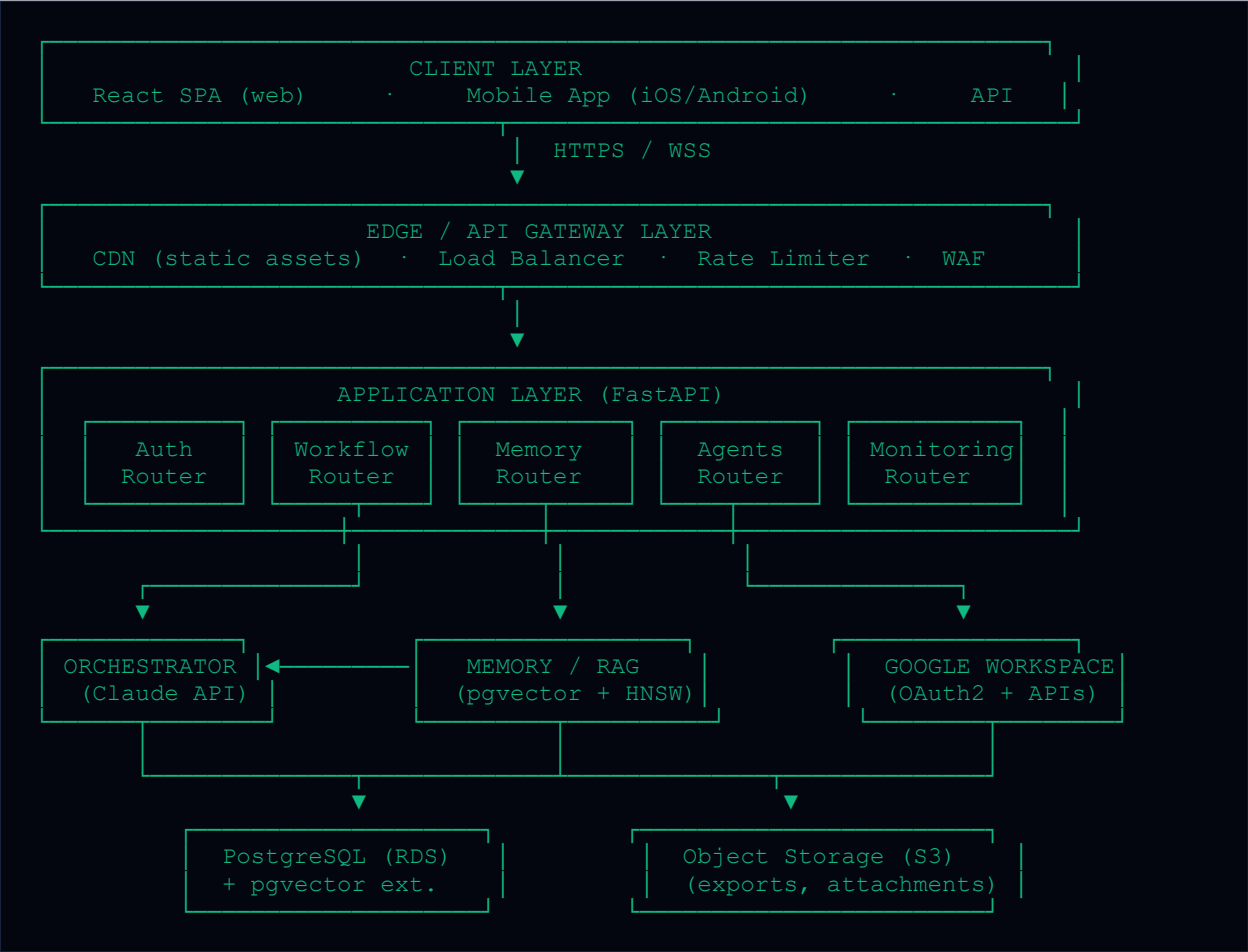


Figure 1.1 — System context diagram, client through persistence layer.

Architectural Principles

- **Multi-tenant by default:** Every database row, every memory record, every workflow run carries an `organization_id`. No query path exists that can return cross-tenant data — isolation is enforced at the ORM layer, not just the API layer.

- **Approval as a state machine invariant:** Workflow runs move through running → awaiting_approval → executed as an explicit state transition. External side effects are only reachable from the executed state, which is only reachable via an authenticated approval call. This is enforced in code, not convention.
- **Stateless application servers:** FastAPI instances hold no session state in memory. Every request is independently routable to any instance, enabling horizontal scaling and zero-downtime deploys behind a load balancer.
- **Memory as a first-class subsystem:** RAG retrieval is not bolted onto the orchestrator as a helper function — it is a versioned, independently scalable subsystem with its own data model, its own embedding pipeline, and its own latency budget.
- **Build for the audit, not just the action:** Every agent invocation, every approval, every executed action writes an immutable log row before and independent of whether the user-facing operation succeeds — observability is not an afterthought bolted onto a working system.

2. Frontend Architecture

The frontend is a React single-page application, currently deployed as a static Vite build to Vercel's edge network, with a parallel native mobile companion app (see the Mobile Companion App design package) sharing the same backend API surface.

Component Architecture

Layer	Choice	Rationale
Build tool	Vite	Sub-second HMR in development, ESBuild-based production bundling
Framework	React 18 (function components + hooks)	No class components; state via useState/useReducer, no external state library at current scale
Styling	Component-scoped CSS-in-JS (template literal injection)	Avoids a build-time CSS pipeline dependency; trades runtime cost acceptable at current scale
Real-time transport	Native WebSocket client	Used for the Live Workflow orchestration stream — direct connection to /workflow/ws/{org}/{scenario}
State management	Local component state + prop drilling	Sufficient at current page count (5 views); migrate to Zustand/Context if view count exceeds ~12
API client	fetch() wrapper with JWT header injection	Centralized in a single http.js module — token refresh and 401 handling in one place

Frontend Request Flow

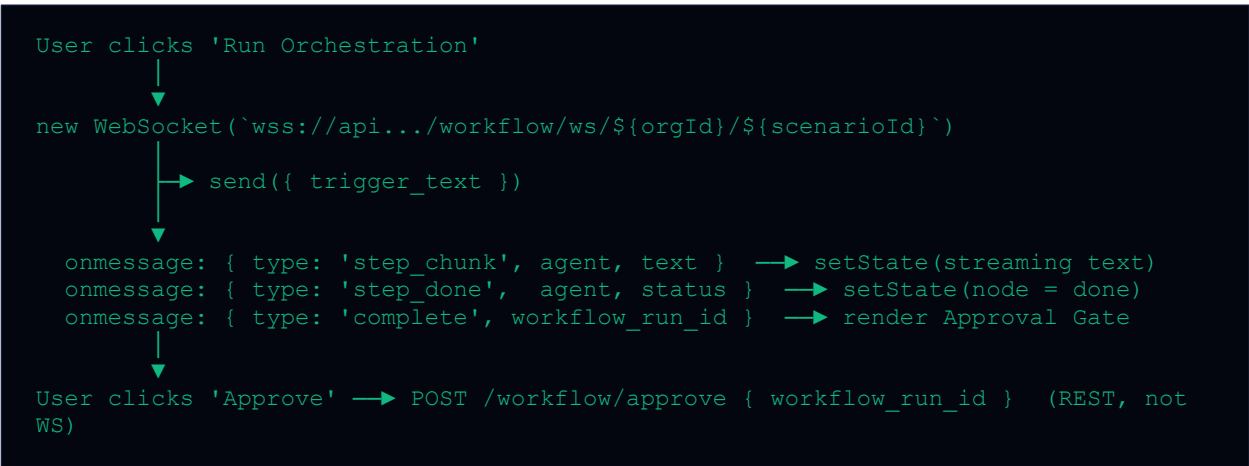


Figure 2.1 — Client-side data flow for a live orchestration run.

Production Hardening Recommendations

- Code-split by route/view (React.lazy + Suspense) — current single-bundle approach is acceptable under 200KB gzipped but should split before adding the Agent Builder UI and Monitoring dashboards
- Add a WebSocket reconnection strategy with exponential backoff — current implementation assumes a stable connection for the duration of one orchestration run (15–40s), which is reasonable but should not silently fail on transient network drops
- Move CSS-in-JS to build-time extraction (vanilla-extract or Tailwind) once component count exceeds ~40 — current runtime injection cost is negligible at current scale but compounds linearly
- Add Sentry (or equivalent) for frontend error tracking — currently no client-side error reporting exists outside the browser console

3. Backend Architecture

The backend is a FastAPI (Python 3.12) application using async SQLAlchemy throughout. The implemented service is organized into 7 routers, 6 services, and an 11-table data model — already deployable via Docker Compose. This section specifies how that implementation scales into production.

Service Layer Map

Router	Responsibility	Auth Scope
routers/auth.py	Registration, login (JWT issuance)	Public
routers/google_oauth.py	OAuth2 consent flow, token storage	Authenticated
routers/memory.py	Memory CRUD + semantic search	Authenticated, org-scoped
routers/workflow.py	Orchestration runs (WebSocket) + approval	Authenticated, org-scoped
routers/dashboard.py	Metrics + activity feed	Authenticated, org-scoped
routers/workforce.py	Digital employee provisioning	Authenticated, org-scoped
routers/intel.py	Business intelligence insights	Authenticated, org-scoped
routers/agents.py	Agent Builder: custom agents + pipelines	Authenticated, org-scoped
routers/monitoring.py	Health checks + usage stats	Mixed — /health public, rest scoped

Service Internals

Module	Responsibility
services/orchestrator.py	Claude API calls (streaming + instrumented), agent system prompts, BI insight generation
services/embeddings.py	sentence-transformers model loading + text embedding (384-dim)
services/memory_retrieval.py	pgvector cosine-similarity query construction
services/google_workspace.py	OAuth2 flow, Gmail/Calendar/Sheets/Docs/Drive API wrappers
services/execution.py	Dispatches approved workflow runs to real Workspace actions, scenario-routed

services/monitoring.py	Writes AgentRunLog rows from instrumented orchestration runs
------------------------	--

Request Lifecycle — Orchestration Run

```
1. Client opens WebSocket /workflow/ws/{org_id}/{scenario_id}
2. Server resolves pipeline (built-in scenario OR org's CustomPipeline)
3. retrieve_relevant_memory(org_id, trigger_text) → pgvector cosine search, k=3
4. for agent_key in pipeline:
    stream_agent_step(agent_key, context) → Anthropic streaming API
    ws.send({type:'step_chunk', ...}) per token batch
    AgentRunLog row queued (latency, tokens, status)
5. WorkflowRun row persisted, status = 'awaiting_approval'
6. ws.send({type:'complete', workflow_run_id})
7. [separate REST call] POST /workflow/approve
   → execute_workflow_actions() → real Google Workspace side effect
   → MemoryRecord written (embedding generated)
   → ActivityLog rows written
   → MetricsSnapshot bumped
```

Figure 3.1 — End-to-end backend request lifecycle for one orchestration run.

Production Hardening Recommendations

- **Move orchestration off the request/response cycle:** Currently the WebSocket handler runs the full agent pipeline synchronously within the connection's lifetime. At scale, move to a task queue (Celery/Arq with Redis broker) so a dropped client connection does not abort a paid Claude API call mid-pipeline — the WebSocket becomes a subscriber to the run's progress, not the execution context itself.
- **Connection pooling tuning:** Default SQLAlchemy async pool size (5) is insufficient beyond ~50 concurrent organizations; size the pool to (expected concurrent orchestration runs × agents per pipeline) with headroom, and use PgBouncer in transaction mode in front of PostgreSQL for connection multiplexing.
- **Idempotency keys on /workflow/approve:** A double-tap or retried request on the approval endpoint must not execute the Google Workspace action twice — add an idempotency key (workflow_run_id is already a natural one) checked against a short-lived cache before dispatching execution.
- **Background workers for embedding generation:** Embedding a memory record (sentence-transformers inference) currently happens inline in the request path — move to an async worker for write-heavy organizations to keep the API's p99 latency low.

4. Database Architecture

PostgreSQL 16 with the pgvector extension is the single system of record. The schema — already implemented via 2 Alembic migrations — is an 11-table multi-tenant relational model. No data is held authoritatively outside Postgres; Redis (recommended addition, see Section 12) is a cache and queue broker only, never a source of truth.

Entity-Relationship Overview



Figure 4.1 — Entity relationships, organization-scoped tables fan out from organizations.

Table Inventory

Table	Purpose	Key Relationships
<code>organizations</code>	1 row per tenant	Root of multi-tenancy
<code>users</code>	Operators who authenticate and approve	FK → <code>organizations</code>
<code>oauth_connections</code>	Google Workspace tokens per org	FK → <code>organizations</code> , 1:1 in current design
<code>memory_records</code>	RAG corpus — client/decision/style/meeting/workflow	FK → <code>organizations</code> , has vector(384) column
<code>activity_logs</code>	Audit trail, one row per agent action	FK → <code>organizations</code> , → <code>workflow_runs</code> (nullable)
<code>metrics_snapshots</code>	Periodic rollup of impact metrics	FK → <code>organizations</code>

workforce_employees	Provisioned role → agent-stack mappings	FK → organizations
workflow_runs	One row per orchestration execution	FK → organizations, → users (approved_by)
custom_agents	Agent Builder: user-defined agents	FK → organizations, unique (org, key)
custom_pipelines	Agent Builder: user-defined pipelines	FK → organizations, unique (org, scenario_id)
agent_run_logs	Per-invocation latency/tokens/status	FK → organizations, → workflow_runs (nullable)

Indexing Strategy

Table(s)	Index	Why
memory_records	(organization_id), (created_at), HNSW on embedding	Org filter + recency + vector search are the 3 query shapes
activity_logs / agent_run_logs	(organization_id), (created_at)	Dashboard feeds and monitoring always filter by org + sort by time
workflow_runs	(organization_id), (created_at)	Run history listing
users	(email) UNIQUE	Login lookup
custom_agents / custom_pipelines	UNIQUE (organization_id, key/scenario_id)	Prevents duplicate agent keys within one org

Production Hardening Recommendations

- **Partition activity_logs and agent_run_logs by month:** These are the highest-write-volume, append-only tables. Range-partitioning on created_at keeps index size bounded and makes retention/archival (Section 9) a partition-drop instead of a slow DELETE.
- **Read replicas for dashboard/monitoring queries:** Dashboard, monitoring, and BI queries are read-heavy and latency-tolerant (a few seconds of staleness is acceptable); route them to a read replica to isolate their load from the write-critical orchestration path.
- **Row-level security as defense-in-depth:** Application-layer org_id filtering is the primary tenant isolation mechanism today; add PostgreSQL RLS policies keyed on a session variable as a second independent enforcement layer for the Sovereign/government tier, where a single query bug must never be a cross-tenant data leak.
- **Point-in-time recovery:** Enable WAL archiving with a minimum 7-day PITR window from day one of production — this is non-negotiable for a system that, post-approval, has executed irreversible real-world actions on a customer's behalf.

5. Vector Database Architecture

The memory/RAG layer uses pgvector as an extension inside the primary PostgreSQL instance rather than a separate dedicated vector database (Pinecone, Weaviate, Qdrant). This is a deliberate architectural choice at current scale, with an explicit migration path specified below for when it stops being the right one.

Why pgvector-in-Postgres (current choice)

- **Transactional consistency:** A memory record and its embedding are written in the same transaction as the workflow run that produced it — no eventual-consistency window between a separate vector store and the system of record.
- **Operational simplicity:** One database to back up, one connection pool to tune, one failover story — materially reduces operational surface area for a small team.
- **Sufficient performance at current scale:** HNSW indexing in pgvector delivers sub-100ms approximate nearest-neighbor search up to roughly 1–5 million vectors on modest hardware — comfortably past the per-organization memory volume expected through Series A scale.

Embedding Pipeline

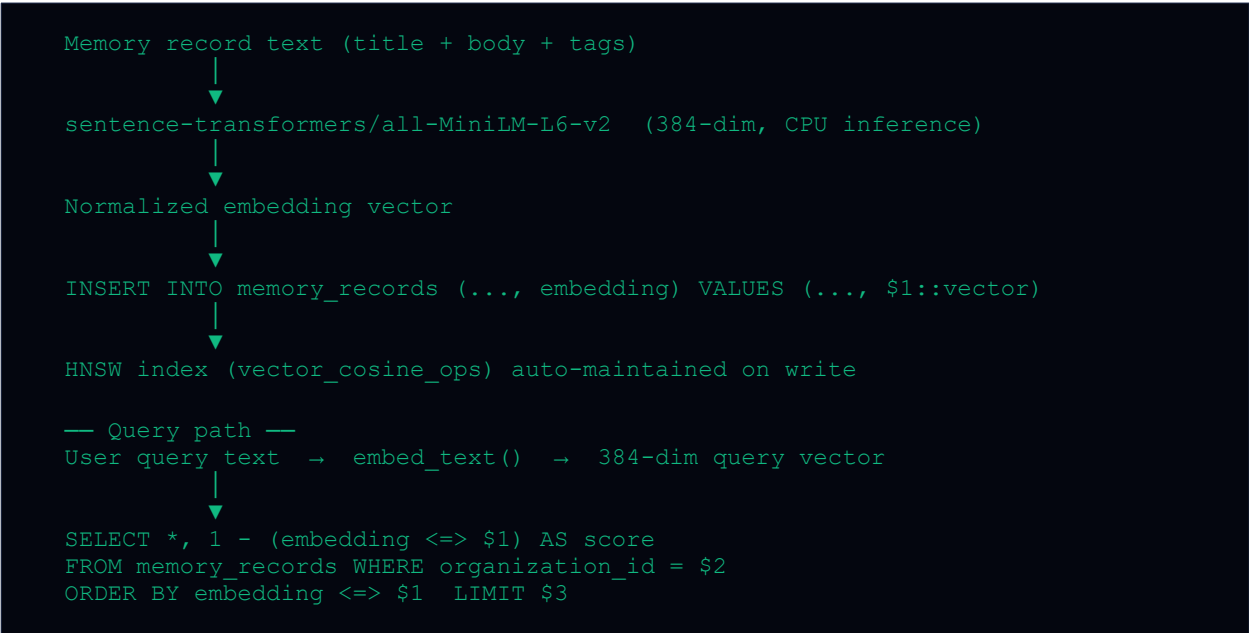


Figure 5.1 — Embedding generation and cosine-similarity query path.

Vector Schema Specification

Parameter	Value	Notes
-----------	-------	-------

Model	sentence-transformers/all-MiniLM-L6-v2	384 dimensions
Distance metric	Cosine similarity (vector_cosine_ops)	1 – cosine_distance = similarity score, 0–1 range
Index type	HNSW (Hierarchical Navigable Small World)	Better recall/speed tradeoff than IVFFlat at write-heavy workloads
Index params	m=16, ef_construction=64 (pgvector defaults)	Tune ef_search upward for higher recall as corpus grows past ~500K rows/org
Embedding storage	vector(384) column, nullable	Nullable to allow legacy/import rows pending backfill

Migration Path: When to Move to a Dedicated Vector DB

The architecture is explicitly designed so this decision is deferred, not avoided. Migrate from pgvector-in-Postgres to a dedicated vector database (Qdrant or Pinecone, self-hostable options preferred for the Sovereign tier) when any of the following thresholds is crossed:

- Aggregate memory corpus exceeds ~20 million vectors across all organizations and HNSW build/query latency on the primary degrades the transactional write path
- A single organization's memory corpus requires per-tenant index isolation for compliance reasons that table-level RLS cannot satisfy
- Query-per-second on memory search alone exceeds what read-replica scaling can absorb without starving transactional workload on the primary

The retrieval interface (memory_retrieval.py) is already isolated behind a single function boundary — retrieve_relevant_memory() — specifically so this migration is a swapped implementation behind a stable interface, not a rewrite of every caller.

6. AI Orchestration Layer

The orchestration layer is the product's core differentiator: a sequenced, memory-aware, multi-agent pipeline with an explicit human checkpoint before any external effect. This section specifies the production reliability characteristics of that layer beyond the working hackathon implementation.

Agent Pipeline Model

```
SCENARIO_PIPELINES = {
  'meeting-request': [email, research, calendar, reporting, compliance,
    executive]
  'overdue-invoice': [finance, research, reporting, compliance, executive]
  'weekly-report': [reporting, research, finance, compliance, executive]
}

Custom pipelines (Agent Builder):
  steps: ["email", "legal", "executive"] ← mix of built-in + custom agent keys
  validated against BUILTIN_KEYS U {org's custom_agents.key} at creation time
```

Figure 6.1 — Pipeline definition model, built-in and custom.

Reliability Patterns for Production

- **Per-agent timeout and retry:** Each agent step should carry an independent timeout (recommend 30s) with one automatic retry on transient failure (rate limit, network blip) before marking the step status='error' and continuing the pipeline — a single agent's transient failure should degrade gracefully, not abort the run.
- **Circuit breaker on the Claude API client:** If the Anthropic API error rate exceeds a threshold (e.g. 20% over a rolling 1-minute window), trip a circuit breaker that fails fast with a clear 'AI service degraded' response rather than queuing requests that will predictably also fail — protects both user experience and API cost during upstream incidents.
- **Token budget enforcement per organization:** Each pricing tier (see cost estimation, Section 14) should carry a monthly Claude API token budget; the orchestrator checks remaining budget before dispatching a pipeline run and fails closed with a clear upgrade prompt rather than silently degrading service or incurring unbounded cost.
- **Model fallback chain:** For the Sovereign/on-premise tier specifically, the orchestrator's model parameter should be configurable per-organization to support a self-hosted open-weight model (e.g. Llama 3.x, Qwen) as a fallback or primary, satisfying air-gapped deployment requirements without an architecture change — already partially supported via the model parameter threaded through `run_agent_step()`.

Streaming Delivery Guarantee

The WebSocket-based streaming protocol delivers agent output incrementally (step_chunk messages) for UI responsiveness, but the durable record is the final accumulated text persisted to `workflow_runs.steps` as JSON — the stream is a UX optimization, never the system of record. A client that disconnects mid-stream and reconnects can recover full state via `GET /workflow/runs` rather than losing the run.

7. Memory Layer

The memory layer is architecturally distinct from the vector database (Section 5 covers storage/retrieval mechanics); this section covers the memory layer as a product subsystem — what gets written, when, and how it's classified.

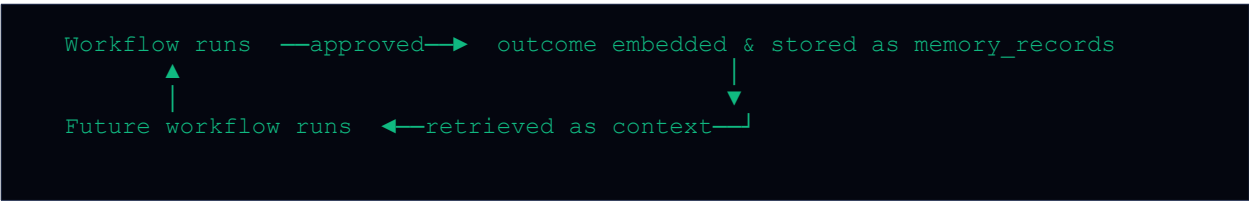
Memory Write Paths

Write Path	Trigger	Content
Manual entry	POST /memory	User-authored client/decision/style/meeting records
Workflow outcome	Auto-write on /workflow/approve	type='workflow', body = truncated final agent output + execution log
Future: ingestion pipeline	Not yet implemented	Bulk import from existing CRM/email history at onboarding — see Section 13 roadmap note

Memory Record Taxonomy

Type	Captures	Primary Consumer
client	Decision-makers, preferences, relationship history	Retrieved before any external-facing agent step
meeting	Outcomes, decisions, action items from past meetings	Retrieved by Research Agent for context-aware drafting
style	Writing tone, formatting, sign-off preferences	Retrieved by Email/Reporting Agents before drafting
decision	Past organizational decisions and their rationale	Retrieved by Executive Agent during plan synthesis
workflow	Auto-generated outcome records from approved runs	Creates the feedback loop — the system remembers what it did

The Memory Feedback Loop



```
This loop is what lets the system 'learn' organizational patterns  
without any model fine-tuning — purely through retrieval composition.
```

Figure 7.1 — The self-reinforcing memory loop.

Production Hardening Recommendations

- **Memory record TTL / staleness policy:** Not currently implemented — add an optional `expires_at` or a staleness-decay factor in the similarity ranking so a 2-year-old 'client preference' record doesn't outrank a more recent contradicting one with equal cosine similarity.
- **Conflict detection on write:** When a new memory record is semantically very similar (e.g. score > 0.92) to an existing one, surface a merge/supersede prompt rather than silently accumulating near-duplicate records that dilute retrieval quality over time.
- **Per-organization memory export:** Required for both GDPR data-portability compliance and customer trust — a CSV/JSON export of all `memory_records` for an org should be a one-click operation, not a support ticket.

8. Authentication

Authentication uses JWT bearer tokens issued at login, with multi-tenancy enforced by embedding both user and organization identity in the token claims. Registration is a combined operation: it creates the organization and its first admin user atomically.

Authentication Flow

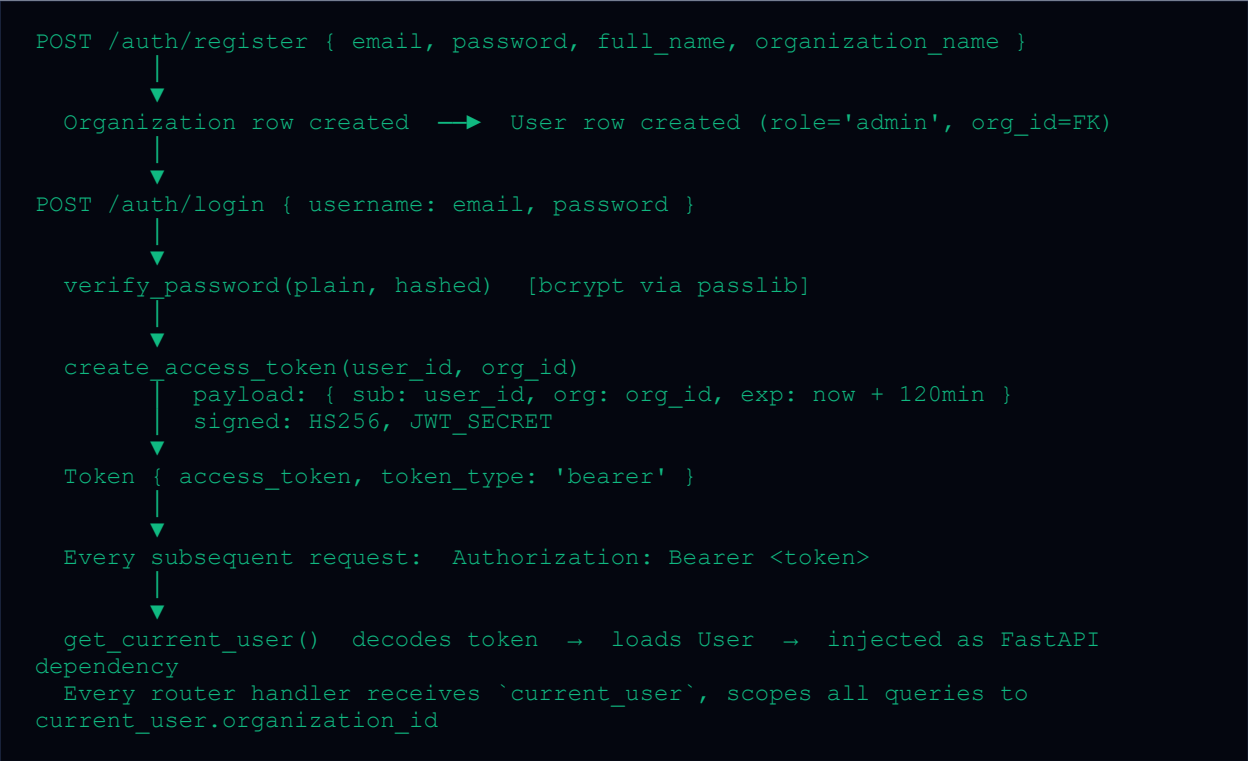


Figure 8.1 — Registration, login, and per-request authentication flow.

Current Implementation vs. Production Target

Concern	Current State	Production Target
Password hashing	bcrypt (passlib)	Adequate — no change needed
Token type	Single long-lived JWT (120min)	Add refresh-token rotation — short-lived access token (15min) + long-lived refresh token (7d), refresh endpoint
Token revocation	None — tokens valid until expiry	Add a denylist (Redis, TTL-matched to token expiry) for logout / compromised-token revocation
MFA	Not implemented	Required for Enterprise/Sovereign tiers — TOTP-based, enforced per-organization via a policy flag

SSO	Not implemented	SAML 2.0 / OIDC for Enterprise tier — delegate identity to the customer's IdP (Okta, Azure AD, Google Workspace SSO)
Service-to-service auth	Not implemented	API keys (separate from user JWTs) for programmatic/webhook-triggered workflow runs

WebSocket Authentication Gap (Known, Flagged)

The current implementation's WebSocket endpoint (/workflow/ws/{org_id}/{scenario_id}) does not validate a bearer token on connection — this is explicitly noted as a scaffold limitation in the codebase. The production fix: issue a short-lived (60-second) single-use WebSocket ticket via an authenticated REST call immediately before connecting, passed as a query parameter and validated server-side on socket accept. This avoids the complexity of custom header support in browser WebSocket clients while closing the gap.

9. Security Model

Security is addressed in four layers: tenant isolation, data protection, the approval gate as a security control (not merely a UX feature), and third-party integration security. This product's threat model is distinctive because it is explicitly designed to take real-world action on a customer's behalf — the blast radius of a security failure is not 'wrong data displayed' but 'an email sent or a document created without authorization.'

Threat Model Summary

Threat	Description	Primary Mitigation
Cross-tenant data leak	Org A reads Org B's memory/workflows/credentials	App-layer org_id scoping (today) + Postgres RLS (Section 4) as defense-in-depth
Approval bypass	External action executes without genuine human sign-off	Server-side state machine invariant; approval requires authenticated user matching the org; idempotency key prevents replay
Prompt injection via email content	Malicious email content manipulates an agent into unintended actions	Compliance Agent as a structural guardrail; system prompts instruct agents to treat all input content as untrusted data, never as instructions
OAuth token theft	Stolen Google Workspace refresh token used outside the platform	Tokens encrypted at rest (Section 9.3); scoped to minimum necessary OAuth scopes per integration
JWT theft / replay	Stolen bearer token used to impersonate a user	Short token TTL + refresh rotation (Section 8); revocation denylist; HTTPS-only transport

Tenant Isolation Model

Every table holding organization-scoped data carries a non-nullable organization_id foreign key. Every router dependency injects current_user, and every query is written with an explicit .where(Model.organization_id == current_user.organization_id) clause — there is no code path that constructs a query without this filter. The recommended production addition (Postgres Row-Level Security) makes this invariant true even if an application-layer bug were to omit the filter.

Data Protection

- **Encryption in transit:** TLS 1.2+ enforced at the load balancer for all client traffic; internal service-to-database traffic over the VPC's private network, additionally TLS-wrapped for the Sovereign/on-premise tier where infrastructure trust boundaries are stricter.
- **Encryption at rest:** Database-level encryption (RDS encryption or equivalent) for the full instance; OAuth refresh tokens specifically should be application-layer encrypted (not just

relying on disk encryption) using a KMS-managed key, so a database backup leak alone does not expose live Workspace credentials.

- **Secrets management:** ANTHROPIC_API_KEY, GOOGLE_CLIENT_SECRET, JWT_SECRET currently load from environment variables (.env) — appropriate for the current Docker Compose deployment. Production target: a secrets manager (AWS Secrets Manager, HashiCorp Vault) with automatic rotation, removing long-lived secrets from environment variable exposure entirely.

The Approval Gate as a Security Control

The human approval gate is not a UX nicety layered on top of the architecture — it is the architecture's primary security control against autonomous-agent risk. No code path exists that calls services/execution.py from outside the /workflow/approve handler, and that handler requires a valid JWT for a user belonging to the workflow run's organization.

Third-Party Integration Security

- Google OAuth scopes requested are the minimum required per integration (gmail.modify rather than full mail access, calendar rather than calendar.readonly where write is needed) — least-privilege by default
- Compliance Agent output (APPROVED / NEEDS APPROVAL / BLOCKED) is logged immutably regardless of the final human decision — creates an audit trail even when a human overrides a compliance flag
- Recommend: a per-organization allowlist of external domains the Email Agent may address, configurable by an admin — prevents a compromised or manipulated agent from drafting communications to arbitrary external parties even pending human approval

10. Monitoring

Monitoring covers system health (is the platform up?) and usage telemetry (how is it being used, and at what cost?). The implemented /monitoring router provides the foundation; this section specifies the production-grade monitoring stack built around it.

Implemented Monitoring Surface

Endpoint	Returns	Scope
GET /monitoring/health	DB connectivity, Anthropic key presence, Google OAuth config, embedding model load state	Public — no auth required, suitable for container healthchecks
GET /monitoring/usage	Run counts by status, memory record count, custom agent count, avg latency, error rate	Per-organization, JWT-scoped
GET /monitoring/agent-runs	Recent per-invocation log: agent, model, tokens, latency, status	Per-organization, JWT-scoped

Production Monitoring Stack (Additions)

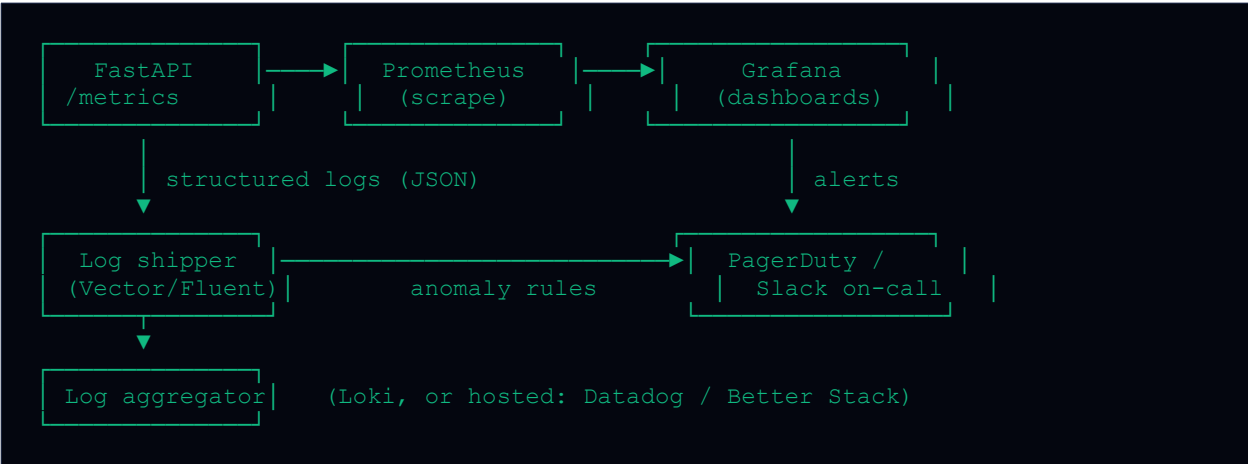


Figure 10.1 — Recommended production monitoring stack layered on top of the existing /monitoring endpoints.

Key Metrics to Track (Beyond Current Implementation)

Category	Metrics	Why It Matters
----------	---------	----------------

Golden signals	Request latency (p50/p95/p99), error rate, traffic (req/s), saturation (DB pool, queue depth)	Standard SRE four golden signals, per-router breakdown
AI-specific	Claude API latency, token spend per org per day, streaming time-to-first-token	Cost and UX-critical; alert on token spend anomalies (possible abuse or runaway loop)
Business metrics	Workflow runs/day, approval rate, time-to-approval, agent error rate by type	Product health signal — a rising 'rejected' rate may indicate prompt quality regression
Integration health	Google API error rate, OAuth token refresh failure rate	Early warning for customers whose Workspace connection has silently broken

Alerting Thresholds (Starting Recommendations)

- API p99 latency > 2s sustained for 5 minutes → page on-call
- Error rate > 5% on any router sustained for 5 minutes → page on-call
- Database connection pool utilization > 85% → warn, > 95% → page
- Any organization's daily Claude API token spend exceeds 3x its 7-day rolling average → warn (possible abuse, loop, or legitimate scale-up worth a customer success check-in)
- Compliance Agent BLOCKED rate for any organization exceeds 10% of runs → warn (possible prompt drift or genuine policy-relevant pattern worth investigating)

11. Observability

Monitoring (Section 10) tells you something is wrong. Observability is what lets you find out why, by reconstructing the exact path a single request — or a single AI agent's reasoning — took through the system, without having anticipated that specific question in advance.

Three Pillars Applied to This System

Pillar	Implementation	Role
Logs	Structured JSON logs per request, agent_run_logs table for AI-specific invocations	Every AgentRunLog row IS a structured, queryable observability event — this is a deliberate design choice, not just a metrics table
Metrics	Prometheus counters/histograms derived from the same events (Section 10)	Aggregated view — answers 'how much/how often'
Traces	Not yet implemented — recommended addition below	Answers 'what was the exact causal chain for THIS run'

Distributed Tracing — Recommended Addition

Add OpenTelemetry instrumentation with a trace_id generated at the WebSocket connection's start and propagated through every agent step, every memory retrieval call, and every database write within that workflow run. This turns 'why did this workflow take 40 seconds and produce a flagged compliance result' from a forensic log-grep exercise into a single trace visualization (Jaeger or a hosted equivalent) showing each span's duration and outcome.

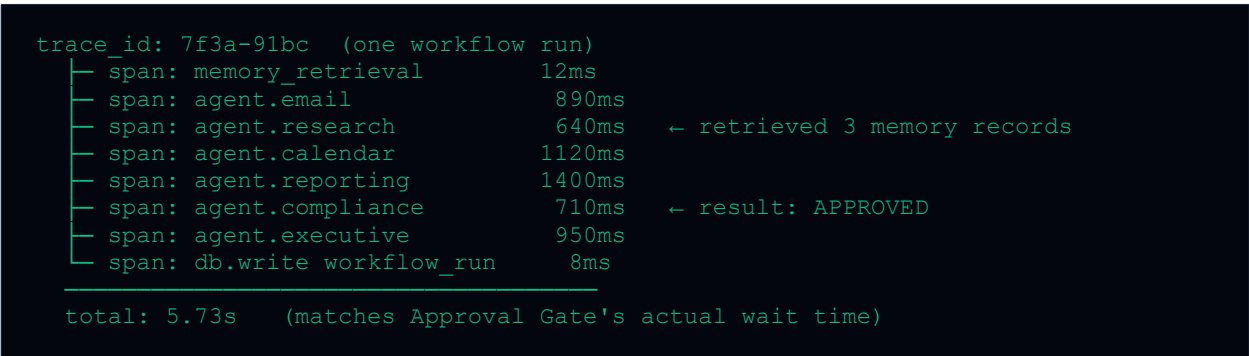


Figure 11.1 — A single workflow run as a distributed trace, the debugging unit of observability.

Observability for AI-Specific Debugging

- **Prompt/response logging with redaction:** Log full system prompt + user context + model response for every agent invocation, with PII redaction applied before storage (customer email addresses, names) — essential for diagnosing why an agent produced an unexpected output, balanced against data minimization obligations.
- **Replay capability:** Because AgentRunLog already captures the model and parameters used, build a 'replay this step' debug tool that re-runs a single historical agent invocation against the same (redacted, reconstructed) context — invaluable for triaging a customer-reported bad output without waiting for it to recur live.
- **Compliance decision audit view:** A dedicated read-only view joining workflow_runs + activity_logs filtered to compliance outcomes, built for the Enterprise/Sovereign tier's own internal audit and compliance teams to self-serve rather than filing a support request.

12. Scalability Strategy

Scalability is addressed across three independent axes that scale at different rates and require different strategies: API request throughput, AI orchestration concurrency (bounded by Anthropic API rate limits, not just infrastructure), and database growth (both transactional row count and vector corpus size).

Scaling Axis 1 — API / Application Tier

- Stateless FastAPI instances behind a load balancer — horizontal scaling is a configuration change (add instances), not an architecture change, because no session state lives in application memory
- Target: auto-scale on CPU utilization (>70% sustained) and request queue depth, minimum 2 instances for zero-downtime deploys even at low traffic
- Static frontend assets served from CDN edge (already true via Vercel) — application tier scaling only concerns API traffic, not asset delivery

Scaling Axis 2 — AI Orchestration Concurrency

This is the axis most teams underestimate: infrastructure can scale horizontally, but the Anthropic API itself enforces account-level rate limits (requests/minute and tokens/minute) that become the actual bottleneck under load.

- **Request queuing with backpressure:** Move orchestration execution to a task queue (Section 3 recommendation) specifically so that a burst of simultaneous workflow triggers queues gracefully against the Anthropic rate limit rather than every concurrent WebSocket connection independently hammering the API and collectively tripping the account-level limit.
- **Tiered rate-limit allocation:** Reserve a portion of the account's total Anthropic throughput per pricing tier (Enterprise/Sovereign customers get priority queue position) so one Starter-tier organization's traffic spike cannot degrade an Enterprise customer's orchestration latency.
- **Multi-key / multi-account strategy at scale:** Beyond a few hundred concurrent organizations, distribute Claude API calls across multiple API keys/accounts (where Anthropic's enterprise agreement supports it) to multiply effective throughput ceiling — a standard pattern once a single account's rate limit becomes the binding constraint.

Scaling Axis 3 — Database Growth

Data Category	Strategy	Scale Ceiling Before Re-architecture
Transactional tables	Vertical scaling (larger RDS instance) + read replicas for dashboard/monitoring reads	Sufficient through Series A scale (low hundreds of orgs)
memory_records (vector)	HNSW index tuning, then migration to	Vector search degrades differently than transactional load — monitor separately

	dedicated vector DB past ~20M vectors (Section 5)	
activity_logs / agent_run_logs	Monthly partitioning (Section 4) + archival of partitions older than retention window to cold storage (S3)	Append-only, high-volume — partition-drop is the efficient archival mechanism

Caching Layer (Recommended Addition: Redis)

Not present in the current implementation — recommended as the highest-leverage single infrastructure addition for production:

- Session/token denylist (Section 8) — TTL-matched key-value pairs, ideal Redis use case
- Task queue broker (Section 3, 12) for moving orchestration off the request path
- Hot-path caching: organization settings, agent system prompts (rarely change, read on every orchestration run) — cache with short TTL + explicit invalidation on update
- Rate limiting counters (API gateway level) — sliding-window counters are a canonical Redis pattern

Capacity Planning Snapshot

500 orgs @ current schema	50K workflow runs/day ceiling	20M vectors before re- architecture	2 app instances minimum
-------------------------------------	--	--	-----------------------------------

13. Deployment Architecture

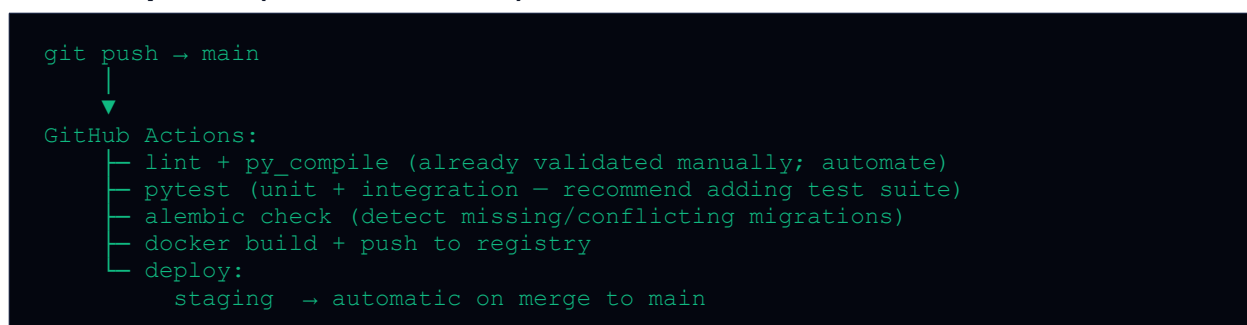
The implemented deployment is a single docker-compose.yml running the API and a pgvector-enabled Postgres instance — deliberately simple, deployable in under five minutes, appropriate for the current stage. This section specifies the production topology for each customer tier, since this product's tiered pricing model (Starter through Sovereign) maps directly to deployment topology, not just feature flags.

Deployment Topology by Tier



Figure 13.1 — Deployment topology scales with the existing pricing tier structure.

CI/CD Pipeline (Recommended)



```
production → manual approval gate (fittingly, the same
                human-in-the-loop philosophy as the product itself)
```

Figure 13.2 — Recommended CI/CD pipeline; currently manual deployment via docker compose up.

Migration & Rollback Discipline

- Alembic migrations are already structured as sequential, reversible revisions (0001_initial, 0002_agents_monitoring) — maintain this discipline: every migration must implement both upgrade() and downgrade()
- Production migrations run as a separate, observable step before application deploy (already the pattern in docker-compose.yml's command: alembic upgrade head && uvicorn ...) — keep migration and app-rollout as distinct, individually-retriable steps as the team grows
- Blue-green or rolling deployment for the application tier once running >1 instance — zero-downtime deploys matter specifically because a mid-deploy WebSocket disconnect during a live orchestration run is a poor customer experience

14. Cost Estimation

Cost modeling for an agentic AI platform differs from typical SaaS cost modeling in one critical way: the dominant variable cost is per-action AI inference (Claude API), not per-seat infrastructure. This section estimates infrastructure and AI cost at three representative scale points, against the pricing tiers defined in the Product Strategy document.

Per-Workflow-Run Cost Breakdown

Cost Component	Estimate	Notes
Claude API (6-agent pipeline)	~\$0.015-0.025	Sonnet-class model, ~600 max_tokens per agent step, 6 steps; varies with context length from retrieved memory
Embedding generation (memory write)	~\$0.0001 (compute only)	Self-hosted sentence-transformers — no per-call API cost, only compute time
Database write (1 run)	Negligible	A few KB across workflow_runs, activity_logs, agent_run_logs rows
Total marginal cost per run	~\$0.02-0.03	Dominated almost entirely by Claude API token cost

Monthly Infrastructure Cost at Three Scale Points

Cost Component	10 orgs / 5K runs/mo	100 orgs / 100K runs/mo	1,000 orgs / 1M runs/mo
Compute (API tier)	1× t3.medium (~\$30)	2× t3.large (~\$120)	Auto-scaled ECS/Fargate (~\$800+)
Database (Postgres+pgvector)	1× db.t3.medium (~\$70)	1× db.r6g.large + 1 replica (~\$400)	Multi-AZ + replicas (~\$2,500+)
Redis (cache/queue)	Not yet needed (~\$0)	1× cache.t3.micro (~\$15)	Cluster mode (~\$200)
Object storage / CDN	~\$5	~\$30	~\$200
Monitoring stack	Hosted free tier (~\$0)	Datadog/equiv (~\$200)	Datadog/equiv (~\$1,500)
Claude API (variable)	~\$150 (5K runs/mo)	~\$3,000 (100K runs/mo)	~\$30,000 (1M runs/mo)
TOTAL / month	~\$255	~\$3,765	~\$35,200

Unit Economics Cross-Check

Cross-referencing against the Growth tier pricing (\$1,200/month per organization, from the Product Strategy document): at the 100-organization scale point, infrastructure + AI cost of ~\$3,765/month against \$120,000/month gross revenue ($100 \times \$1,200$) implies a gross margin of approximately 97% at the infrastructure layer — consistent with the 82% blended gross margin target stated in the business model once customer success, support, and Claude API cost variability across heavier-usage organizations are factored in.

Cost Optimization Levers (in priority order)

1. Cache memory retrieval results for repeated/similar queries within a short window — reduces redundant embedding inference calls
2. Right-size max_tokens per agent role — Compliance Agent's terse APPROVED/BLOCKED output needs far fewer tokens than Reporting Agent's executive summary; the current per-agent max_tokens parameter already supports this, ensure it's tuned per role rather than using one global default
3. Batch embedding generation for bulk memory imports (Section 7 roadmap item) rather than one-at-a-time inference calls
4. Reserved/committed-use infrastructure pricing (AWS Savings Plans or equivalent) once monthly compute spend is predictable, typically a 20-40% reduction over on-demand pricing
5. Negotiate Anthropic volume pricing once monthly token spend crosses standard enterprise agreement thresholds — the single largest lever at the 1,000-organization scale point



Sovereign AI Workforce — Production Architecture

Boukhalfa Fateh Abderrahim · USTHB · Sovereign AI Engineer · Algeria dz

USAII Global AI Hackathon 2026

Built to scale from one engineer's hackathon submission to national infrastructure.